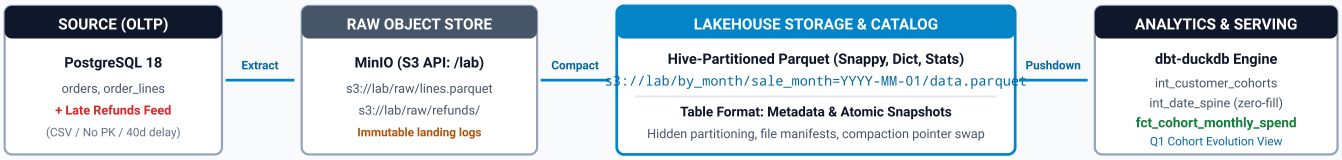
Checkpoint 1: Storage Layer Design

QUESTION 1: CUSTOMER COHORT SPEND EVOLUTION OVER TIME

Big Data: Persistence Systems & Languages
Team Q01 · FEAA SDBIS (2026–2027)
Prigoreanu Alin-Gabriel, Bîrgovan A. Mihai-Daniel, Schipor Ana-Maria

1. STORAGE LAYER ARCHITECTURE & DATA PIPELINE FLOW



2. PARTITION KEY ARITHMETIC & SIZING PROJECTIONS (1× VS. 100× SHOP)

Candidate Key	1× Distinct Folders	1× Rows / File	1× Actual Size	100× Scale Rule	100× Folders	100× Est. File Size	Verdict for Question 1
sale_month	**36 folders**	**55,555 rows**	**1.20 MiB**	**Static (36 calendar months)**	**36 files**	**116.6 MiB**	**Chosen: Fits 100–512 MB sweet spot; natural cohort grain.**
category	6 folders	333,333 rows	7.16 MiB	Static product taxonomy	6 files	716.0 MiB	**Rejected:** Too coarse; orthogonal to cohort timeline.
city	16 folders	125,000 rows	2.69 MiB	Static delivery hubs	16 files	269.0 MiB	**Rejected:** Geographic partition; zero cohort pruning benefit.
sale_day	1,096 folders	1,824 rows	55.5 KiB	Static (3 years × 365 days)	1,096 files	3.8 MiB	**Rejected:** Severe small-file problem; 30× request overhead.
customer_id	119,975 folders	16.7 rows	0.36 KiB	Scales with revenue (×100)	12,000,000 files	0.36 KiB	**Rejected:** Massive metadata explosion; paralyzes S3 listing.

3. FOUR CORE ARCHITECTURAL DECISIONS FOR QUESTION 1

1. File Format — PARQUET (SNAPPY)

Choice: Columnar Parquet with dictionary encoding and statistics (min/max) in row group footers for all analytical reads.

Rejected Alternative: Uncompressed or gzip CSV/JSON lines directly on object storage.

Reason from Q1: Cohort spend analysis requires scanning only 4 columns across millions of rows: `customer_id`, `created_at`, `quantity`, and `unit_price`. Parquet's projection pushdown avoids fetching unused transaction fields (shipping, addresses, notes), cutting network payload from MinIO by over 70%.

2. Partitioning Strategy — BY SALE_MONTH (36 FILES)

Choice: Hive-style directory partitioning on transaction month: `by_month/sale_month=YYYY-MM-01/data_0.parquet`.

Rejected Alternatives: Partitioning on `customer_id` (12M micro-files) or no partitioning (one single 4.1 GiB file).

Trade-Off from Q1: *Speeds up* monthly cohort spend recalculations and late-month auditing via partition pruning (reading 1 or 12 files instead of 36). *Slows down* cross-time queries like customer lifetime lookups or city-wide historical aggregations, which must pay 36 separate GET requests for footers.

3. File Sizing & Compaction — 116.6 MIB TARGET / COMPACTION

Choice: Target 100 MiB to 256 MiB per partition file. For the 100× shop, 36 months over 4.1 GiB yields exactly **116.6 MiB/file**.

Mitigating Small Files: Ingestion batches (daily extracts or hourly micro-batches) write temporary delta files. A scheduled daily compaction job rewrites them using `COPY ... PARTITION_BY (sale_month) ORDER BY created_at, order_line_id` into consolidated monthly Parquet files.

Reason from Q1: Preserves dense row groups (100k+ rows) and prevents the 76× request explosion measured in W5 step 5.

4. Table Format (Iceberg / DuckLake) — TABLE FORMAT WARRANTED

Choice: Adopt a modern table format with metadata manifests, atomic commits, and snapshot isolation over raw folder paths.

Rejected Alternative: Plain object storage folder listing (globbing `*.parquet`).

Reason from Q1: Crucial for two reasons demonstrated in W4 & W5: (1) **Late Refunds (up to 40 days delay):** When March refunds arrive in May, historical cohort revenue must update retroactively. Table formats provide ACID snapshot isolation so readers never read corrupt half-written files. (2) **Compaction:** Atomically swaps file pointers from old delta files to compacted files without duplicating data.